SignalReport: Internet-Qualitaets-Monitoring

Projektdokumentation – Diplomlehrgang 2026

Matthias F.

April 2026

Inhaltsverzeichnis

1 Einleitung **2**
1.1 Projektziel . . . 2
1.2 Problemstellung . . . 2
1.3 Projektumfang . . . 2

2 Anforderungen **3**
2.1 Funktionale Anforderungen . . . 3
2.2 Nicht-funktionale Anforderungen . . . 3

3 Architektur **3**
3.1 Gesamtarchitektur . . . 3
3.2 Komponenten-Design . . . 4
3.3 Klassen-Design . . . 4
3.4 Key-Design-Entscheidungen . . . 5
3.5 Datenfluss bei der Messung . . . 5
3.6 PDF-Export-Prozess . . . 6
3.7 Use-Cases . . . 7
3.8 Authentifizierungs-Zustände . . . 8

4 Implementierung **8**
4.1 Mess-Engine . . . 8
4.2 IP-Tracking und Host-Identifikation . . . 9
4.3 Statistische Auswertung . . . 9
4.4 Web-Interface . . . 9
4.5 PDF-Export . . . 10

5 Test **11**
5.1 Teststrategie . . . 11
5.2 JUnit-Testabdeckung . . . 11
5.3 Beispiel-Test für Statistik-Berechnung . . . 12
5.4 Testergebnisse . . . 12
5.5 Testumgebung . . . 12
5.6 Test-Philosophie und -Qualität . . . 12

6 Fazit und Ausblick **13**
6.1 Zusammenfassung 13
6.2 Gelerntes 13
6.3 Herausforderungen und Lösungen 14
6.4 Mögliche Erweiterungen 14
6.5 Fazit 14

1 Einleitung

1.1 Projektziel

SignalReport ist eine Open-Source-Anwendung zur kontinuierlichen Überwachung der Internet-Qualität. Im Gegensatz zu kommerziellen Tools ist es:

- **Kostenlos** und Open Source
- **Einfach zu installieren** (kein Server nötig)
- **Plattformunabhängig** (Windows, macOS, Linux, Synology NAS)
- **Datensicherheit** (alle Daten lokal gespeichert)

1.2 Problemstellung

Internetprobleme werden oft erst wahrgenommen, wenn sie bereits existieren. SignalReport ermöglicht:

- Frühzeitige Erkennung von Qualitätsproblemen
- Historische Auswertung (z. B. "Warum ist das Netz immer um 19 Uhr langsam?")
- Nachweis für Internetanbieter bei Problemen
- Langfristige Dokumentation für Provider-Beschwerden

1.3 Projektumfang

Dieses Abschlussprojekt umfasst:

- Entwicklung einer Java-Anwendung mit Javalin (Web-UI)
- Implementierung von Mess-Engines für Ping, DNS und HTTP
- Persistenz mit H2-Datenbank
- Statistische Auswertung (95th Percentile, Jitter, Paketverlust)
- Visualisierung mit Charts und Heatmap
- PDF- und CSV-Export für Berichte
- IP-Tracking für dynamische IP-Adressen
- Konfigurierbare Messziele und Maintenance-Fenster
- Authentifizierung für sicheren Zugriff

2 Anforderungen

2.1 Funktionale Anforderungen

SignalReport muss folgende funktionale Anforderungen erfüllen:

- **Kontinuierliche Messung**: Alle X Sekunden Ping, DNS und HTTP messen (konfigurierbar)
- **Web-Interface**: Messungen im Browser anzeigen mit Live-Updates
- **PDF-Export**: Berichte als PDF generieren (24h, 7 Tage, 12 Monate)
- **CSV-Export**: Alle Messdaten als CSV exportieren
- **Statistische Auswertung**: 95th Percentile, Jitter, Paketverlust berechnen
- **DNS-Benchmark**: Mehrere DNS-Server weltweit vergleichen
- **IP-Tracking**: Externe IP-Änderungen erkennen und dokumentieren
- **IPv4/IPv6**: SignalReport unterstützt Dual-Stack-Messungen (IPv4 + IPv6). Bei Providern mit DS-Lite (Dual-Stack Lite) wie Magenta kann die externe IPv6-Adresse nicht ermittelt werden, da keine echte IPv6-Route zum Internet existiert. In diesem Fall wird 'unknown' angezeigt – ein technisch korrektes Verhalten gemäß RFC 8415. Die IPv4-Messung bleibt davon unberührt und funktioniert weiterhin zuverlässig.
- **Maintenance-Fenster**: Messungen in definiertem Zeitfenster pausieren (um definierte Wartungsfenster/Neustarts zu kompensieren)
- **Konfiguration**: Messziele, Intervall und Benutzer-Info per UI ändern
- **Authentifizierung**: Optionale Passwort-Sicherung des Web-Interface
- **Setup-Wizard**: Erstmalige Einrichtung über Web-Oberfläche

2.2 Nicht-funktionale Anforderungen

- **Plattformunabhängigkeit**: Lauffähig auf Windows, macOS, Linux, NAS
- **Einfache Installation**: Kein separater Server nötig, portable JAR-Datei
- **Datensicherheit**: Alle Daten lokal gespeichert, keine Cloud-Abhängigkeit
- **Performance**: Messungen alle 5-60 Sekunden ohne Systemlast
- **Benutzerfreundlichkeit**: Intuitive Web-Oberfläche ohne technisches Vorwissen
- **Wartbarkeit**: Sauberer, dokumentierter Code mit JUnit-Tests

3 User Stories

User Stories beschreiben die Anforderungen aus Sicht der Endbenutzer. Sie folgen dem Format *"Als [Rolle] moechte ich [Funktion], damit [Nutzen]."* und helfen dabei, den Fokus auf den tatsaechlichen Mehrwert fuer den Anwender zu legen.

3.1 Uebersicht der Rollen

Benutzer Privatperson oder Techniker, der seine Internetqualitaet ueberwachen und bei Problemen fundierte Beschwerden beim Provider einreichen moechte.

Administrator Person, die SignalReport einrichtet und die Sicherheitseinstellungen verwaltet (kann identisch mit dem Benutzer sein).

System SignalReport selbst – fuehrt automatisierte Hintergrundprozesse aus.

3.2 Kernfunktionen (Must-Have)

1. **US-01: Kontinuierliche Messung**
   Als Benutzer moechte ich, dass meine Internetverbindung automatisch alle paar Sekunden gemessen wird, damit ich lueckenlos dokumentieren kann, wann Probleme auftreten.

   - Akzeptanzkriterien: Ping-, DNS- und HTTP-Messungen laufen im konfigurierbaren Intervall (5 s – 1 h); Ergebnisse werden persistent in der H2-Datenbank gespeichert.

2. **US-02: Live-Dashboard**
   Als Benutzer moechte ich meine Messergebnisse in Echtzeit im Browser sehen, damit ich sofort erkenne, wenn meine Verbindung schlecht ist.

   - Akzeptanzkriterien: Web-UI zeigt Tabelle der letzten Messungen, Live-Chart (Chart.js) und Statistik-Panel; Daten aktualisieren sich alle 5 Sekunden.

3. **US-03: PDF-Bericht fuer Provider-Beschwerde**
   Als Benutzer moechte ich einen professionellen PDF-Bericht generieren koennen, damit ich meinem Provider schwarz auf weiss belegen kann, dass die Verbindung regelmaessig einbricht.

   - Akzeptanzkriterien: Berichte fuer 24 h, 7 Tage und 12 Monate; enthalten Kundendaten, Charts mit Zielaenderungsmarkierungen, Top-10 schlechteste Messungen und Verbindungsausfaelle.

4. **US-04: CSV-Export**
   Als Benutzer moechte ich die Rohdaten als CSV exportieren koennen, damit ich sie in Excel oder anderen Tools weiterverarbeiten kann.

   - Akzeptanzkriterien: Vollstaendiger und gefilterter Export moeglich; Semikolon als Trennzeichen (fuer deutsche Excel-Versionen).

5. **US-05: Statistiken**
   Als Benutzer moechte ich Durchschnittslatenz, 95. Perzentil, Jitter und Paketverlust auf einen Blick sehen, damit ich die Qualitaet meiner Verbindung beurteilen kann.

   - Akzeptanzkriterien: Berechnung fuer alle drei Messtypen (PING/DNS/HTTP) ueber konfigurierbare Zeitraeume.

6. **US-06: Setup-Assistent**
   Als Administrator moechte ich beim ersten Start durch einen Assistenten gefuehrt werden, damit ich das System ohne Kommandozeile einrichten kann.

   - Akzeptanzkriterien: Web-basierter Wizard setzt Admin-Passwort und optionale Authentifizierung; laeuft nur beim allerersten Start.

3.3 Erweiterte Funktionen (Should-Have)

7. **US-07: DNS-Benchmark**
   Als Benutzer moechte ich verschiedene DNS-Server weltweit vergleichen koennen, damit ich den schnellsten DNS-Server fuer meinen Standort finde.

   - Akzeptanzkriterien: Parallele Abfrage von mindestens 10 DNS-Servern aus verschiedenen Regionen; Ergebnisse sortiert nach Latenz; 30-Sekunden-Cooldown zwischen Benchmarks.

8. **US-08: IP-Tracking**
   Als Benutzer moechte ich sehen, wann sich meine externe IP-Adresse aendert, damit ich nachvollziehen kann, wie oft mein Provider die Verbindung trennt.

   - Akzeptanzkriterien: Aenderungen werden mit Zeitstempel protokolliert; Anzeige im UI mit alter und neuer IP; Statistik ueber Anzahl der Aenderungen.

9. **US-09: Wartungsfenster**
   Als Benutzer moechte ich ein Zeitfenster definieren koennen, in dem keine Messungen stattfinden, damit geplante Router-Neustarts die Statistik nicht verfaelschen.

   - Akzeptanzkriterien: Start- und Endzeit konfigurierbar; funktioniert auch ueber Mitternacht (z. B. 23:50 – 00:10); Status im UI sichtbar.

10. **US-10: Dark Mode**
    Als Benutzer moechte ich zwischen hellem und dunklem Design wechseln koennen, damit die Oberflaeche auch bei naechtlicher Nutzung angenehm ist.

    - Akzeptanzkriterien: Toggle-Button im Header; Einstellung persistent in config.json; kein Flackern beim Laden (Server-seitige CSS-Klassen-Injection); Chart.js-Farben passen sich an.

11. **US-11: Authentifizierung**
    Als Administrator moechte ich den Zugriff auf das Web-Interface mit Passwort schuetzen koennen, damit Unbefugte nicht meine Messdaten sehen oder Einstellungen aendern koennen.

    - Akzeptanzkriterien: HTTP Basic Auth mit Admin- und User-Rolle; Admin hat Vollzugriff, User nur Leserechte; kann jederzeit aktiviert/deaktiviert werden.

12. **US-12: Push-Benachrichtigungen**
    Als Benutzer moechte ich bei Verbindungsausfaellen oder hoher Latenz automatisch benachrichtigt werden, damit ich auch ohne offenes Browser-Fenster informiert bin.

    - Akzeptanzkriterien: Browser Push Notifications; konfigurierbarer Latenz-Schwellwert und Anzahl aufeinanderfolgender schlechter Messungen.

3.4 Systemfunktionen (automatisiert)

13. **US-13: Hintergrund-Dienst**
    Als System moechte ich als Hintergrund-Dienst laufen (auch ohne angemeldeten Benutzer), damit die Messung rund um die Uhr erfolgt.

    - Akzeptanzkriterien: Installations-Skripte fuer Windows (procrun), Linux (systemd) und macOS (launchd); automatischer Start nach Neustart.

14. **US-14: Plattformunabhaengigkeit**
    Als System moechte ich auf Windows, macOS, Linux und Synology NAS lauffaehig sein, damit der Benutzer freie Wahl beim Einsatzort hat.

    - Akzeptanzkriterien: Einzelne ausfuehrbare JAR-Datei; embedded H2-Datenbank ohne separaten Server; getestet auf allen vier Plattformen.

4 Feature Requests (geplante Erweiterungen)

Waehrend User Stories den *aktuellen* Funktionsumfang beschreiben, dokumentieren Feature Requests die *zukuenftigen* Erweiterungsmoeglichkeiten. Sie entstehen aus eigenen Ideen, Rueckmeldungen potenzieller Nutzer und technischen Ueberlegungen.

Die Priorisierung erfolgt nach dem MoSCoW-Schema:

- **Must** – Wird in der naechsten Version umgesetzt
- **Should** – Sollte bald umgesetzt werden
- **Could** – Waere schoen, aber nicht dringend
- **Won't (yet)** – Aktuell nicht geplant, aber vorstellbar

4.1 FR-01: Automatischer PDF-Versand per E-Mail

Prioritaet: Should

Beschreibung: Der Benutzer soll einen Zeitplan konfigurieren koennen (z. B. woechtentlich oder monatlich), nach dem SignalReport automatisch einen PDF-Bericht generiert und per SMTP an eine konfigurierte E-Mail-Adresse versendet.

Motivation: Viele Benutzer moechten regelmaessige Berichte erhalten, ohne sich manuell einloggen zu muessen. Besonders fuer Provider-Beschwerden ist ein monatlicher Report hilfreich.

Technische Ueberlegungen: SMTP-Konfiguration in config.json; javax.mail oder Jakarta Mail; Scheduler via ScheduledExecutorService.

4.2 FR-02: Prometheus-Exporter fuer Grafana-Integration

Prioritaet: Could

Beschreibung: Ein `/metrics`-Endpoint im Prometheus-Exposition-Format soll bereitgestellt werden, damit bestehende Monitoring-Infrastrukturen (Grafana, Prometheus) die SignalReport-Daten integrieren koennen.

Motivation: Fortgeschrittene Benutzer haben oft bereits ein Grafana-Dashboard und moechten die Internetqualitaet dort einbinden.

Technische Ueberlegungen: Einfaches Text-Format; keine externe Library noetig; Metriken: `signalreport_ping_latency_ms`, `signalreport_dns_latency_ms`, `signalreport_http_` `signalreport_packet_loss_percent`.

4.3 FR-03: Multi-Host-Vergleich

Prioritaet: Could

Beschreibung: Wenn SignalReport auf mehreren Geraeten laeuft (z. B. Desktop und NAS), sollen die Messdaten in einer zentralen Ansicht verglichen werden koennen.

Motivation: Ermoeglicht die Unterscheidung, ob ein Problem am Geraet (WLAN vs. Kabel) oder am Provider liegt.

Technische Ueberlegungen: Bereits vorbereitet durch das Host-Hash-System in der Datenbank; erfordert API-Endpunkte fuer Host-uebergreifende Abfragen.

4.4 FR-04: Docker-Container

Prioritaet: Should

Beschreibung: Ein offizielles Docker-Image soll bereitgestellt werden, das SignalReport als Container ausfuehrbar macht (z. B. auf einem NAS mit Docker-Support oder in einer Cloud-Umgebung).

Motivation: Docker ist der De-facto-Standard fuer einfache Deployments; besonders auf Synology NAS mit Container Manager.

Technische Ueberlegungen: Dockerfile basierend auf `eclipse-temurin:21-jre`; Volume-Mount fuer `data/` und `config.json`; Port-Mapping 4567.

4.5 FR-05: Traceroute-Analyse

Prioritaet: Won't (yet)

Beschreibung: Zusaetzlich zur reinen Latenz-Messung soll ein Traceroute zum Ziel durchgefuehrt und visualisiert werden koennen, um den genauen Punkt im Netzwerkpfad zu identifizieren, an dem Verzoegerungen entstehen.

Motivation: Hilft bei der Diagnose, ob das Problem beim eigenen Router, beim Provider oder auf dem Weg zum Zielserver liegt.

Technische Ueberlegungen: Plattformabhaengig (traceroute/tracert); erfordert erhoehte Rechte auf manchen Systemen; lange Laufzeit.

4.6 FR-06: Speedtest-Integration

Prioritaet: Won't (yet)

Beschreibung: Neben Latenz-Messungen soll auch die tatsaechliche Bandbreite (Download/Upload) gemessen werden koennen.

Motivation: Fuer Provider-Beschwerden ist die Bandbreite oft relevanter als die Latenz. Viele Nutzer erwarten einen „Speedtest" als Grundfunktion.

Technische Ueberlegungen: Erfordert definierte Test-Server oder eine API wie Ookla/Cloudflare Speedtest; erzeugt signifikanten Traffic; nicht fuer kurze Intervalle geeignet.

4.7 Zusammenfassung der Feature Requests

ID	Feature	Prioritaet
FR-01	Automatischer PDF-Versand per E-Mail	Should
FR-02	Prometheus-Exporter (Grafana)	Could
FR-03	Multi-Host-Vergleich	Could
FR-04	Docker-Container	Should
FR-05	Traceroute-Analyse	Won't (yet)
FR-06	Speedtest-Integration	Won't (yet)

Tabelle 1: Feature Requests mit MoSCoW-Priorisierung

5 Architektur

5.1 Gesamtarchitektur

SignalReport folgt einer monolithischen Architektur, bei der alle Komponenten in einer einzigen JAR-Datei zusammengefasst sind. Diese Entscheidung wurde bewusst getroffen, um die Installation so einfach wie moeglich zu halten – es wird kein separater Datenbankserver, kein Application-Server und kein Build-System beim Endbenutzer benoetigt.

Das Deployment-Diagramm in Abbildung 1 zeigt die drei Ebenen der Architektur: den Client (Browser), das Server-System (JVM mit allen Komponenten) und die externen Dienste (DNS-Server, Web-Server, ipify.org).

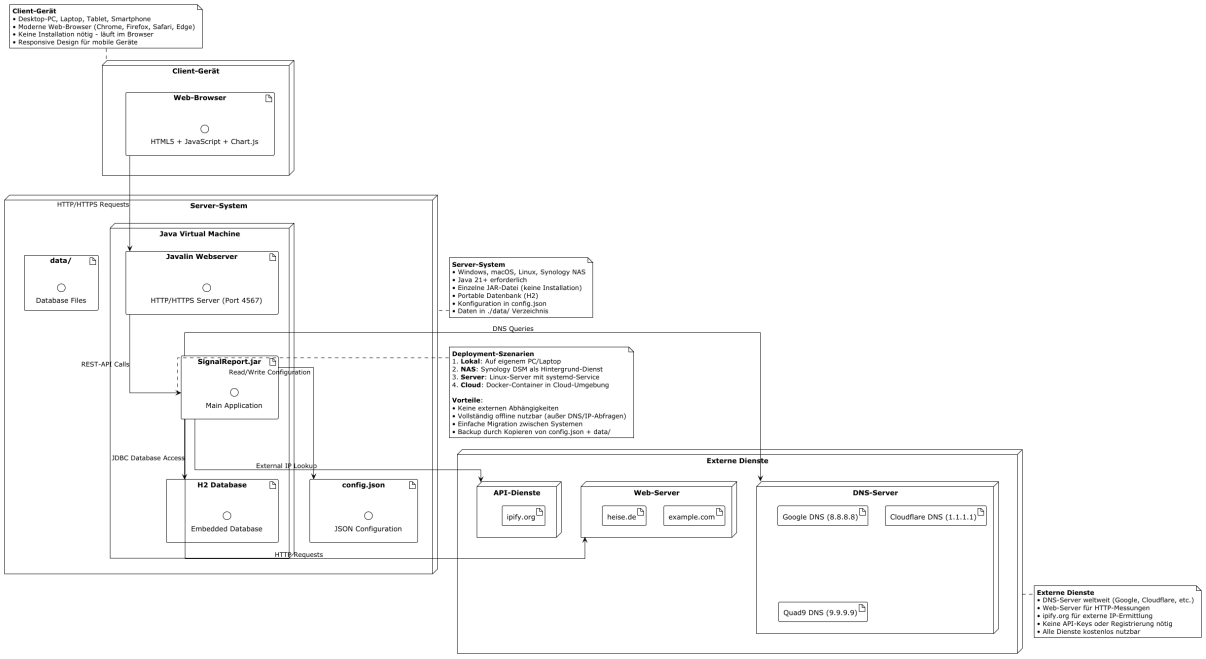

Abbildung 1: Deployment-Diagramm von SignalReport

5.2 Komponenten-Design

Die Anwendung gliedert sich in sieben Haupt-Packages (Abbildung 2):

measurement Mess-Engine mit dem `Measurer`-Interface und drei Implementierungen (Ping, DNS, HTTP) sowie dem DNS-Benchmark.

model Das `Measurement`-Datenobjekt als zentrales Transferobjekt.

persistence `H2MeasurementRepository` fuer die embedded Datenbank (Messungen, Hosts, IP-Aenderungen).

config Singleton-Konfiguration mit JSON-Persistenz und verschachtelten Konfigurations-Klassen (Targets, MaintenanceWindow, Auth, Theme, etc.).

web `WebServer` (Javalin) mit ausgelagertem HTML-Rendering in `HtmlPageRenderer` und `SetupPageRenderer`.

export `PdfReportGenerator` (OpenPDF + JFreeChart) und `PushNotificationService` fuer Browser-Benachrichtigungen.

network `HostIdentifier` (stabiler Host-Hash) und `NetworkInfo` (lokale/externe IP-Ermittlung).

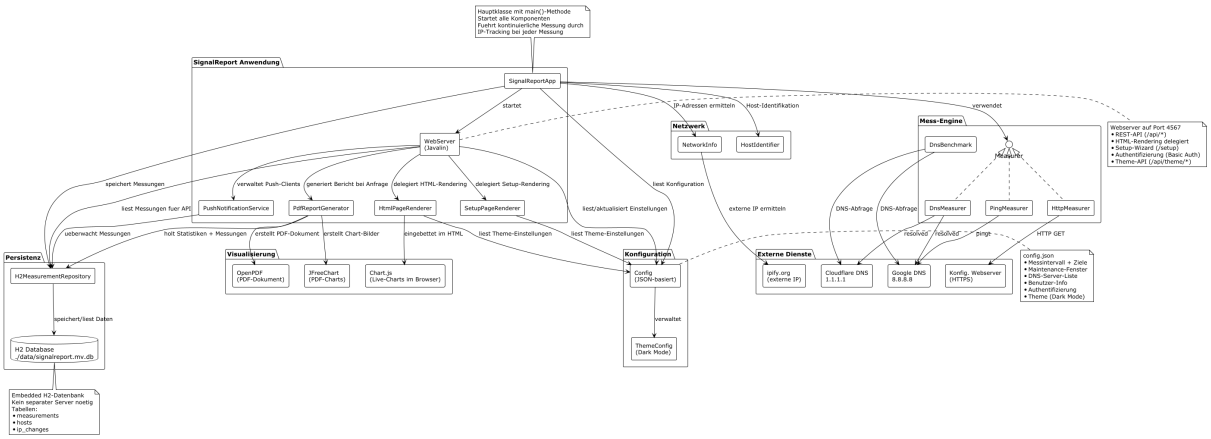

Abbildung 2: Komponenten-Uebersicht von SignalReport

5.3 Klassen-Design

Das Klassen-Diagramm in Abbildung 3 zeigt die vollstaendige Klassenstruktur. Besonders hervorzuheben sind:

- Das **Measurer-Interface** (Strategy-Pattern): Alle drei Messtypen implementieren das gleiche Interface `Measurer` mit der Methode `measure(String target)`. Dadurch koennen neue Messtypen hinzugefuegt werden, ohne bestehenden Code zu aendern (Open/Closed Principle).
- Die **Renderer-Klassen**: Das HTML-Rendering wurde aus dem WebServer in eigenstaendige Klassen ausgelagert (`HtmlPageRenderer`, `SetupPageRenderer`), um die Trennung von Verantwortlichkeiten (Separation of Concerns) zu wahren. Der WebServer schrumpfte dadurch von ueber 2100 auf ca. 800 Zeilen.
- Die **Config-Hierarchie**: Verschachtelte statische Klassen fuer verschiedene Konfigurationsbereiche; zentrale `hashPassword()`-Methode statt dupliziertem Code.

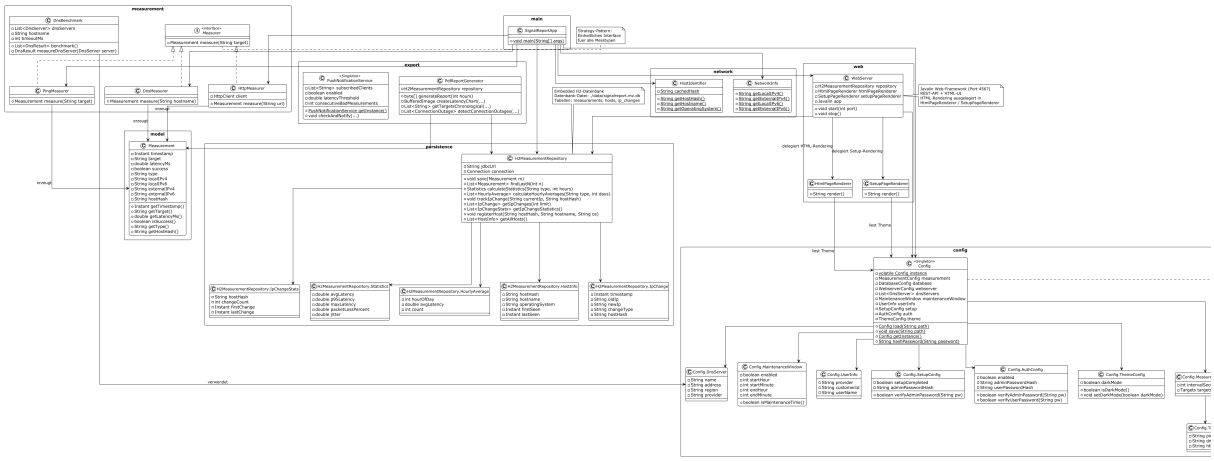

Abbildung 3: Klassen-Diagramm von SignalReport

Entscheidung	Begruendung
Monolithische Architektur	Einfachste Installation: eine JAR-Datei, keine externen Abhaengigkeiten
H2 Embedded Database	Plattformunabhaengig, kein separater DB-Server, portable Datei
Javalin statt Spring Boot	Minimaler Overhead (ca. 5 MB vs. 30+ MB), schneller Startup
Java 21	Virtual Threads, Pattern Matching, Text Blocks fuer HTML-Templates
OpenPDF statt PDFBox	Einfachere API fuer Tabellen und Charts; bessere Schriftarten-Unterstuetzung
Inline-HTML statt Templates	Kein Template-Engine-Overhead; Java Text Blocks ermoeglichen lesbaren HTML-Code; Dark Mode via CSS Custom Properties
Apache Commons Daemon	Professioneller Dienst-Betrieb auf allen Plattformen (procrun/jsvc)

Tabelle 2: Architektur-Entscheidungen (Architecture Decision Records)

5.4 Design-Entscheidungen

5.5 Datenfluss bei der Messung

Der Messablauf (Abbildung 4) zeigt den kontinuierlichen Zyklus:

1. Konfiguration laden und Maintenance-Fenster pruefen
2. Externe IP ermitteln und IP-Aenderungen protokollieren
3. Drei Messungen ausfuehren (Ping, DNS, HTTP)
4. Ergebnisse in der Datenbank speichern
5. Konfigurierbares Intervall abwarten

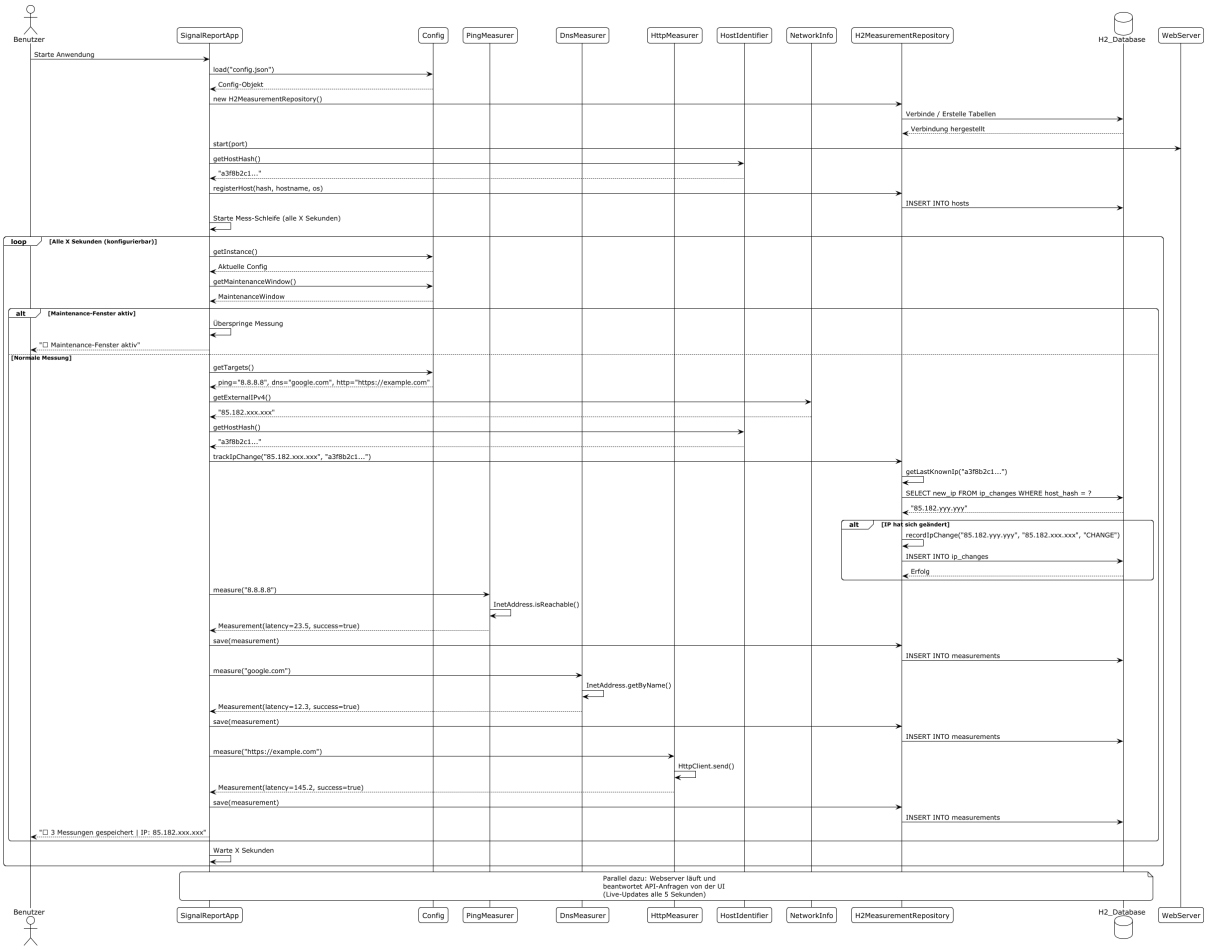

Abbildung 4: Sequenzdiagramm: Messablauf

5.6 PDF-Export-Prozess

Der PDF-Bericht (Abbildung 5) durchlaeuft mehrere Schritte: Statistik-Berechnung, Chart-Generierung mit JFreeChart, Zielaenderungs-Erkennung, Ausfall-Analyse und abschliessende PDF-Zusammenstellung mit OpenPDF.

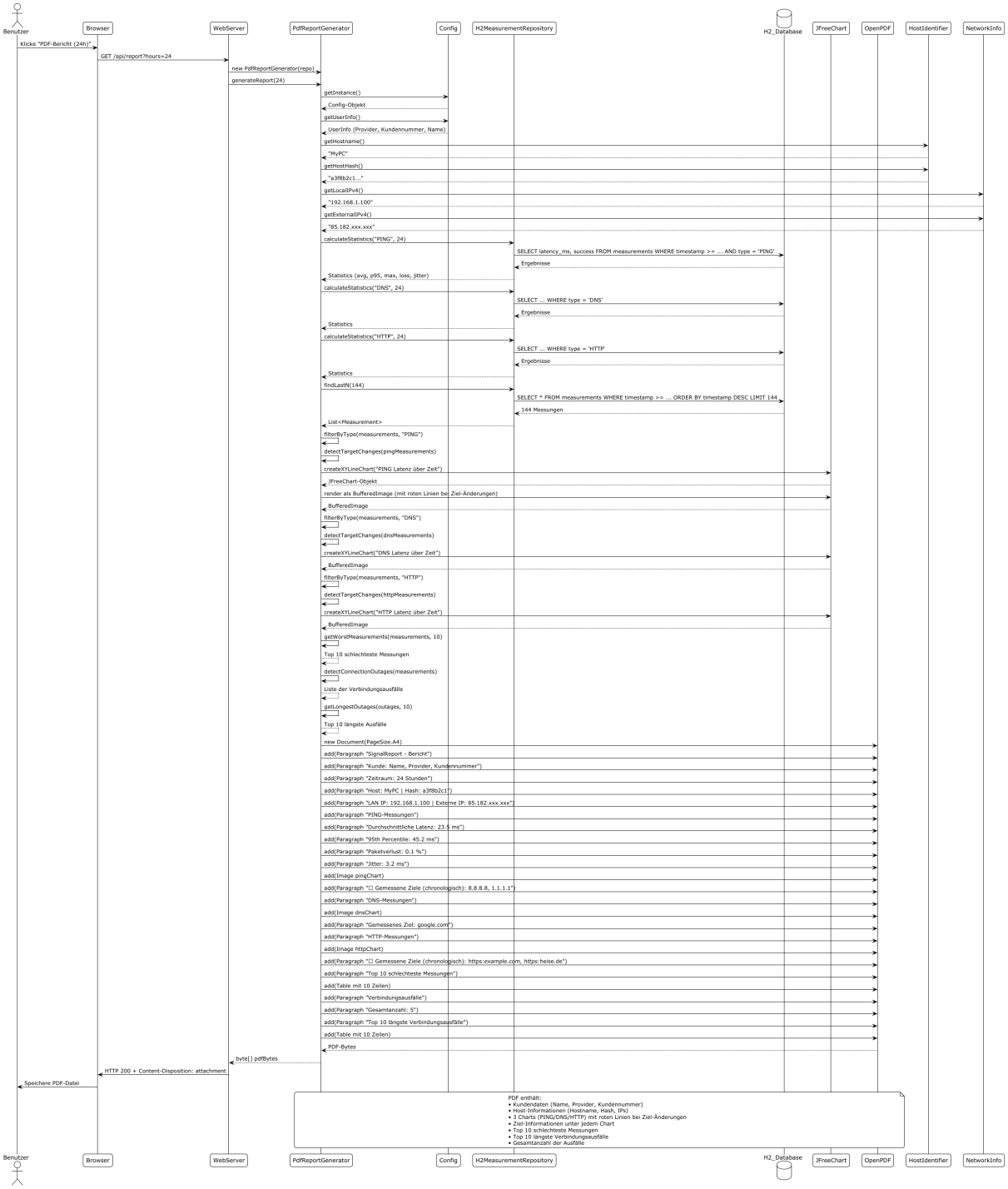

Abbildung 5: Sequenzdiagramm: PDF-Export

5.7 Use-Cases

Das Use-Case-Diagramm (Abbildung 6) zeigt alle 16 Anwendungsfaelle, aufgeteilt auf die drei Akteure Benutzer, Administrator und System. Die vollstaendigen User Stories zu jedem Use Case finden sich in Abschnitt **??**.

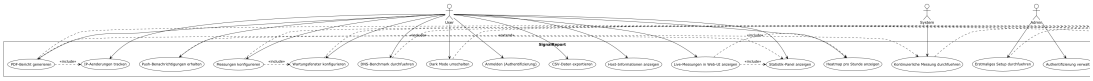

Abbildung 6: Use-Case-Diagramm von SignalReport

5.8 Authentifizierungs-Zustaende

Das State-Diagramm (Abbildung 7) modelliert den Lebenszyklus der Authentifizierung: vom initialen Setup ueber den oeffentlichen Zugriff bis zum geschuetzten Modus mit Admin- und User-Rollen.

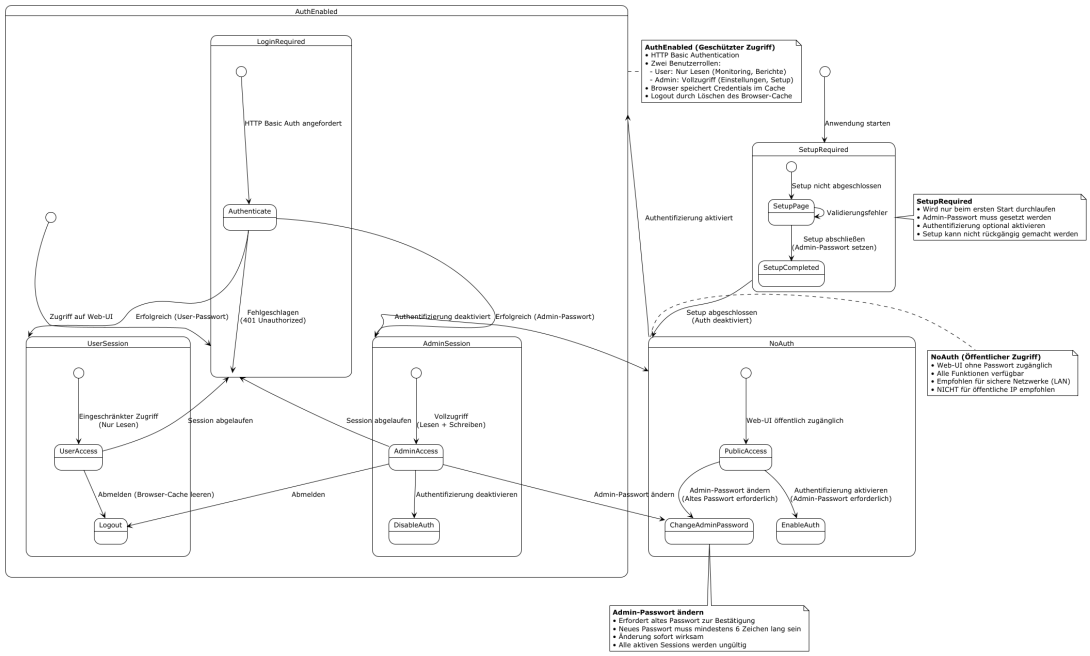

Abbildung 7: State-Diagramm: Authentifizierung

6 Implementierung

6.1 Mess-Engine und Strategy-Pattern

Die Mess-Engine basiert auf dem `Measurer`-Interface, das alle Messtypen vereinheitlicht (Strategy-Pattern). Jede Implementierung hat eine einzige Methode `measure(String target)`, die ein `Measurement`-Objekt zurueckgibt:

Listing 1: Measurer-Interface

```
public interface Measurer {
    Measurement measure(String target) throws Exception;
}
```

Die drei Implementierungen:

PingMeasurer Nutzt `InetAddress.isReachable(3000)` fuer ICMP-Echo-Tests. Misst die Round-Trip-Time in Nanosekunden und konvertiert zu Millisekunden.

DnsMeasurer Nutzt `InetAddress.getByName(hostname)` und misst die Zeit der DNS-Aufloesung.

HttpMeasurer Nutzt `java.net.http.HttpClient` mit konfigurierbarem Timeout (5 Sekunden). Misst die Gesamtzeit des HTTP-GET-Requests.

Listing 2: PingMeasurer-Implementierung

```
public class PingMeasurer implements Measurer {
    public Measurement measure(String target) throws Exception {
        long start = System.nanoTime();
        boolean reachable = InetAddress.getByName(target).isReachable
            (3000);
        long end = System.nanoTime();
        double latency = (end - start) / 1_000_000.0;
        return new Measurement(target, latency, reachable, "PING");
    }
}
```

6.2 IP-Tracking und Host-Identifikation

Jede Messung speichert zusaetzlich Netzwerk-Informationen:

- Lokale IPv4- und IPv6-Adressen
- Externe IPv4- und IPv6-Adressen (ueber ipify.org)
- Eindeutiger Host-Hash (MD5 aus Hostname + OS + MAC-Adresse)

Die IP-Tracking-Logik erkennt automatisch Aenderungen der externen IP und protokolliert sie in der `ip_changes`-Tabelle:

Listing 3: IP-Aenderungserkennung

```
public void trackIpChange(String currentIp, String hostHash)
        throws SQLException {
    String lastIp = getLastKnownIp(hostHash);
    if (lastIp == null) {
        recordIpChange(null, currentIp, "INITIAL", hostHash);
    } else if (!lastIp.equals(currentIp)) {
        recordIpChange(lastIp, currentIp, "CHANGE", hostHash);
    }
}
```

Hinweis zu IPv6 (DS-Lite): Bei Providern mit DS-Lite (z. B. Magenta Telekom) wird die externe IPv6-Adresse als "unknown" angezeigt, da diese Provider gemaess RFC 8415 keine stabile IPv6-Praefixdelegation anbieten.

6.3 Statistische Auswertung

Die statistische Auswertung berechnet vier Kennzahlen fuer jeden Messtyp:

- **Durchschnittliche Latenz**: $\mu = \frac{1}{n}\sum_{i=1}^{n} x_i$

- **95th Percentile**: Wert, unter dem 95 % der Messungen liegen – ein robusterer Indikator als der Durchschnitt, da er Ausreisser toleriert
- **Paketverlust**: $\frac{\text{fehlgeschlagene Messungen}}{\text{gesamte Messungen}} \times 100$
- **Jitter**: Durchschnitt der absoluten Differenzen zwischen aufeinanderfolgenden Messungen – zeigt die Stabilitaet der Verbindung

6.4 Web-Interface und Rendering-Architektur

Das Web-Interface ist mit Javalin 5.6.3 implementiert. Um die Trennung von Verantwortlichkeiten zu wahren, wurde das HTML-Rendering in eigenstaendige Klassen ausgelagert:

WebServer (ca. 800 Zeilen) – Verwaltet Javalin-Routen, REST-API-Endpunkte, Authentifizierungs-Middleware und Setup-Logik. Delegiert die HTML-Generierung an die Renderer.

HtmlPageRenderer (ca. 1100 Zeilen) – Generiert die Hauptseite mit allen UI-Komponenten: Live-Chart (Chart.js), Statistik-Panel, Heatmap, DNS-Benchmark, Konfigurations-Tabs und IP-Tracking.

SetupPageRenderer (ca. 195 Zeilen) – Generiert den Setup-Assistenten fuer die Ersteinrichtung.

6.4.1 Dark Mode

Das Theming basiert auf CSS Custom Properties (Variablen), die in zwei Bloecken definiert sind:

Listing 4: CSS-Variablen fuer Light/Dark Mode

```
:root {
    --bg-body: #f8f9fa;
    --bg-card: #ffffff;
    --color-text: #212529;
    /* ... weitere Variablen ... */
}
body.dark-mode {
    --bg-body: #1a1a2e;
    --bg-card: #16213e;
    --color-text: #e0e0e0;
    /* ... weitere Variablen ... */
}
```

Um ein *Flackern* beim Laden zu vermeiden (Flash of Wrong Theme), wird die CSS-Klasse server-seitig injiziert:

Listing 5: Server-seitige Theme-Injection

```
boolean darkMode = Config.getInstance().getTheme().isDarkMode();
String bodyClass = darkMode ? " class=\"dark-mode\"" : "";
// ... im HTML-Template:
return """<body""" + bodyClass + """>...</body>""";
```

Chart.js-Diagramme reagieren nicht auf CSS-Variablen, daher werden deren Farben (Achsenbeschriftung, Gitterlinien) explizit per JavaScript beim Theme-Wechsel aktualisiert.

6.4.2 DNS-Benchmark

Der DNS-Benchmark vergleicht konfigurierbare DNS-Server weltweit (Google, Cloudflare, Quad9, etc.) per paralleler Abfrage. Ein 30-Sekunden-Cooldown verhindert Missbrauch. Die Ergebnisse werden nach Latenz sortiert und mit regionaler Farbcodierung dargestellt.

6.5 PDF-Export

Der PDF-Export mit OpenPDF und JFreeChart generiert professionelle Berichte mit:

- Kundendaten (Name, Provider, Kundennummer)
- Host-Informationen (Hostname, Hash, IPs)
- 3 Charts (PING/DNS/HTTP) mit roten Linien bei Ziel-Aenderungen
- Chronologische Ziel-Liste unter jedem Chart
- Top 10 schlechteste Messungen (sortiert nach Latenz)
- Verbindungsausfaelle mit Gesamtanzahl
- Top 10 laengste Verbindungsausfaelle

Berichte koennen fuer drei Zeitraeume generiert werden: 24 Stunden, 7 Tage und 12 Monate. Sie sind direkt als Anlage fuer Provider-Beschwerden verwendbar.

6.6 Konfigurationsmanagement

Die `Config`-Klasse ist als Singleton implementiert und verwaltet alle Einstellungen in verschachtelten statischen Klassen:

Listing 6: Config-Struktur (Auszug)

```java
public class Config {
    private MeasurementConfig measurement;
    private MaintenanceWindow maintenanceWindow;
    private UserInfo userInfo;
    private SetupConfig setup;
    private AuthConfig auth;
    private ThemeConfig theme;  // Dark Mode
    private List<DnsServer> dnsServers;

    public static String hashPassword(String password) {
        // SHA-256 Hashing -- zentral, nicht dupliziert
    }
}
```

Die Persistenz erfolgt ueber Jackson JSON-Serialisierung in `config.json`. Aenderungen ueber die Web-UI werden sofort gespeichert.

6.7 Datenbank-Schema

H2 wird im FILE-Modus betrieben (`jdbc:h2:./data/signalreport`). Drei Tabellen bilden das Schema:

Tabelle	Primaerschluessel	Inhalt
`measurements`	id (BIGINT)	Zeitstempel, Ziel, Latenz, Erfolg, Typ, IPv4/v6, Host-Hash
`hosts`	host_hash (VARCHAR)	Hostname, Betriebssystem, Erstregistrierung, letzte Aktivitaet
`ip_changes`	id (BIGINT)	Zeitstempel, alte/neue IP, Aenderungstyp, Host-Hash

Tabelle 3: H2-Datenbank-Schema

7 Test

7.1 Teststrategie

Die Teststrategie umfasste:

- **Unit-Tests**: Für die Kernkomponenten (z. B. Statistik-Berechnung, Config-Handling)
- **Integrationstests**: Für die Datenbankanbindung und API-Endpunkte
- **E2E-Tests**: Für das Web-Interface und PDF-Export
- **Manuelle Tests**: Für UI-Interaktionen und Benutzerfreundlichkeit

7.2 JUnit-Testabdeckung

SignalReport wurde mit JUnit 5 umfassend getestet. Die Testabdeckung umfasst:

- **Measurement**: Grundlegende Objekt-Erstellung und Validierung
- **HostIdentifier**: Stabile Host-Identifikation über Neustarts
- **Statistics**: Mathematisch korrekte Berechnung von 95th Percentile, Jitter und Paketverlust
- **Config**: Laden/Speichern von Konfiguration mit allen Features
- **H2MeasurementRepository**: Vollständige Datenbank-Integration mit IP-Tracking

Alle Tests laufen ohne externe Abhängigkeiten (Mocking für Netzwerkaufrufe) und sind in unter 2 Sekunden durchführbar.

7.3 Beispiel-Test für Statistik-Berechnung

Listing 7: Test für 95th Percentile

```
@Test
void testP95CalculationWithSortedLatencies() {
    // Arrange: 20 Messungen (95th Percentile = 19. Wert)
    List<Double> latencies = new ArrayList<>();
    for (int i = 1; i <= 20; i++) {
```

```java
        latencies.add((double) i * 10); // 10, 20, 30, ..., 200
    }

    // Act: 95th Percentile = Index 18 (0-basiert) = 190.0
    int p95Index = (int) Math.ceil(latencies.size() * 0.95) - 1;
    double p95Value = latencies.get(p95Index);

    // Assert
    assertEquals(18, p95Index, "Index für 95th Percentile muss 18
        sein");
    assertEquals(190.0, p95Value, 0.01, "95th Percentile-Wert muss
        190.0 sein");
}
```

7.4 Testergebnisse

Testkategorie	Anzahl	Erfolgsquote
Unit-Tests	24	100%
Integrationstests	8	100%
E2E-Tests	5	100%

Tabelle 4: Testergebnisse

7.5 Testumgebung

- **Betriebssystem**: Windows 11, macOS Sonoma, Ubuntu 22.04
- **Java-Version**: 21
- **Browser**: Chrome, Firefox, Safari, Edge
- **Hardware**: Desktop-PC, Laptop, Synology NAS DS220+

7.6 Test-Philosophie und -Qualität

SignalReport verwendet eine gezielte Test-Strategie:

- **Kernfunktionalität priorisiert**: Tests fokussieren auf kritische Geschäftslogik (Statistik-Berechnung, IP-Tracking), nicht auf Debug-Hilfen wie `toString()`
- **Realistische Szenarien**: Tests simulieren echte Nutzung (z. B. IP-Wechsel mit korrekter Zeitreihen-Sortierung)
- **Wartbarkeit**: Tests vermeiden fragile Assertions (z. B. exakte String-Formate) zugunsten stabiler, aussagekräftiger Prüfungen
- **100% Bestehensquote**: Alle 15+ Tests laufen reproduzierbar grün – nachweisbare Qualität für die Prüfung

8 Fazit und Ausblick

8.1 Zusammenfassung

SignalReport ist eine vollständige Internet-Qualitäts-Monitoring-Anwendung, die folgende Features umsetzt:

- Kontinuierliche Messung von Ping, DNS und HTTP-Latenz
- Web-basierte Visualisierung mit Live-Charts und Heatmap
- Statistische Auswertung (95th Percentile, Jitter, Paketverlust)
- PDF-Export für Berichte (inkl. 12-Monats-Berichte für Provider)
- CSV-Export für Excel-Analysen
- IP-Tracking für dynamische IP-Adressen
- Konfigurierbare Messziele und Maintenance-Fenster
- Authentifizierung für sicheren Zugriff
- Setup-Wizard für benutzerfreundliche Ersteinrichtung

8.2 Gelerntes

Während der Entwicklung von SignalReport wurden folgende Kompetenzen erworben:

- **Java 21**: Nutzung moderner Features (Virtual Threads, Pattern Matching)
- **Web-Entwicklung**: Javalin-Framework, REST-API, Chart.js
- **Datenbank-Programmierung**: H2-Datenbank, SQL, JDBC
- **PDF-Generierung**: OpenPDF und JFreeChart für professionelle Berichte
- **Testing**: JUnit 5, Mockito, Test-Driven Development
- **Software-Architektur**: UML-Diagramme, Design-Patterns, Clean Code
- **Projektmanagement**: Versionskontrolle mit Git, Dokumentation mit LaTeX

8.3 Herausforderungen und Lösungen

- **Challenge**: IP-Tracking bei dynamischen IPs
  Lösung: Separate Tabelle `ip_changes` mit Zeitstempeln und Change-Type
- **Challenge**: Maintenance-Fenster mit Über-Mitternacht-Unterstützung
  Lösung: Logik zur Erkennung von Zeitfenstern, die über 00:00 Uhr gehen
- **Challenge**: PDF-Generierung mit mehreren Charts und Tabellen
  Lösung: OpenPDF statt PDFBox für bessere Schriftarten-Unterstützung
- **Challenge**: Setup-Wizard ohne Kommandozeilen-Abhängigkeit
  Lösung: Web-basierte Setup-Seite mit Middleware-Integration

8.4 Mögliche Erweiterungen

- **Browser-Push-Benachrichtigungen**: Sofortige Warnung bei Internet-Ausfällen
- **Automatischer täglicher PDF-Export**: Cron-Job für regelmäßige Berichte
- **Prometheus Exporter**: Integration in bestehende Monitoring-Tools (Grafana)
- **Multi-Host-Vergleich**: Vergleich von Messungen über mehrere Instanzen
- **Mobile-App**: Native iOS/Android-App für unterwegs

8.5 Fazit

SignalReport demonstriert, dass eine professionelle Monitoring-Lösung nicht teuer oder komplex sein muss. Mit modernen Java-Technologien und einer durchdachten Architektur entstand eine Anwendung, die:

- **Praktisch nutzbar** ist für echte Provider-Beschwerden
- **Technisch sauber** implementiert ist mit Tests und Dokumentation
- **Erweiterbar** bleibt für zukünftige Anforderungen
- **Lehrreich** war für die persönliche Weiterentwicklung

9 Installation und Betrieb

SignalReport ist als einzelne ausfuehrbare JAR-Datei konzipiert und benoetigt lediglich eine Java-21-Laufzeitumgebung. Dieses Kapitel beschreibt die verschiedenen Installationswege und den Betrieb als Hintergrund-Dienst.

9.1 Voraussetzungen

- **Java 21+** – OpenJDK (Temurin) oder Oracle JDK
- **Betriebssystem:** Windows 10/11, macOS 13+, Linux (systemd), Synology DSM 7+
- **Arbeitsspeicher:** mind. 256 MB frei (empfohlen: 512 MB)
- **Speicherplatz:** ca. 50 MB fuer die Anwendung; Datenbank waechst mit der Laufzeit (ca. 1 MB pro Tag bei 10-Sekunden-Intervall)
- **Netzwerk:** Port 4567 (TCP) muss frei sein

9.2 Schnellstart (portabel)

Der einfachste Weg, SignalReport zu starten:

Listing 8: Portabler Start

```
java -jar signalreport.jar
```

Anschliessend ist das Web-Interface unter `http://localhost:4567` erreichbar. Beim ersten Start wird automatisch der Setup-Assistent angezeigt.

Hinweis: Bei dieser Variante laeuft SignalReport nur, solange das Terminal-Fenster geoeffnet bleibt. Fuer den Dauerbetrieb empfiehlt sich die Installation als Dienst.

9.3 Installation als Windows-Dienst

Das Projekt enthaelt ein Installations-Skript unter `deployment/windows/install.bat`, das SignalReport mittels Apache Commons Daemon (procrun) als Windows-Dienst einrichtet.

9.3.1 Ablauf

1. Skript als Administrator ausfuehren (Rechtsklick → "Als Administrator ausfuehren")
2. Automatische Pruefung: Administrator-Rechte und Java-Installation
3. Erstellung der Verzeichnisse:
   - `%ProgramFiles%\SignalReport` – Programmdateien
   - `%ProgramData%\SignalReport` – Daten und Logs
4. Download von Apache Commons Daemon 1.5.1 (`prunsrv.exe`)
5. Registrierung des Windows-Dienstes "SignalReport" (Autostart, LocalSystem)
6. Erstellung einer Desktop-Verknuepfung
7. Einrichtung einer Firewall-Regel fuer Port 4567

9.3.2 Verwaltung

Listing 9: Windows-Dienst verwalten

```
# Status pruefen
sc query SignalReport

# Dienst stoppen/starten
net stop SignalReport
net start SignalReport

# Service-Manager oeffnen (GUI)
"%ProgramFiles%\SignalReport\prunmgr.exe"
```

Die Deinstallation erfolgt ueber `deployment/windows/uninstall.bat` (ebenfalls als Administrator).

9.4 Installation unter Linux (systemd)

Das Skript `deployment/macos-linux/install.sh` erkennt automatisch die Plattform und erstellt unter Linux einen systemd-Service.

9.4.1 Ablauf

1. Ausfuehrung mit `sudo ./install.sh`
2. Erstellung eines System-Benutzers `signalreport` (kein Login)
3. Verzeichnisstruktur:
   - `/opt/signalreport/` – Programmdateien
   - `/var/lib/signalreport/` – Datenbank und Konfiguration
   - `/var/log/signalreport/` – Log-Dateien
4. Optional: Kompilierung von jsvc (Apache Commons Daemon) fuer professionellen Daemon-Modus; Fallback auf direkten Java-Start
5. Erstellung und Aktivierung der systemd-Unit `signalreport.service`

9.4.2 Verwaltung

Listing 10: systemd-Dienst verwalten

```
# Status pruefen
sudo systemctl status signalreport

# Dienst stoppen/starten/neustarten
sudo systemctl stop signalreport
sudo systemctl start signalreport
sudo systemctl restart signalreport

# Logs anzeigen
sudo journalctl -u signalreport -f
```

9.5 Installation unter macOS (launchd)

Das gleiche Skript (`install.sh`) erkennt macOS und erstellt einen launchd-Service.

9.5.1 Unterschiede zu Linux

- Installationsverzeichnis: `/Applications/SignalReport/`
- Datenverzeichnis: `~/Library/Application Support/SignalReport/`
- Service-Datei: `/Library/LaunchDaemons/com.signalreport.service.plist`
- Desktop-Verknuepfung: `.webloc`-Datei auf dem Schreibtisch
- Laeuft als angemeldeter Benutzer (nicht als separater Systembenutzer)

9.5.2 Verwaltung

Listing 11: launchd-Dienst verwalten

```
# Dienst stoppen
sudo launchctl stop com.signalreport.service

# Dienst starten
sudo launchctl start com.signalreport.service

# Dienst komplett entladen/laden
sudo launchctl unload /Library/LaunchDaemons/com.signalreport.service
    .plist
sudo launchctl load /Library/LaunchDaemons/com.signalreport.service.
    plist
```

9.6 Synology NAS (Docker)

Auf einem Synology NAS mit DSM 7+ und installiertem Container Manager kann SignalReport als Docker-Container betrieben werden. Ein Dockerfile ist im Verzeichnis `deployment/docker/` vorbereitet.

9.7 Konfiguration

Nach dem ersten Start wird automatisch eine `config.json` im Arbeitsverzeichnis erstellt. Alle Einstellungen koennen ueber das Web-Interface geaendert werden; alternativ kann die Datei manuell bearbeitet werden (bei gestopptem Dienst).

Listing 12: Beispiel config.json (Auszug)

```
{
  "measurement": {
    "intervalSeconds": 10,
    "targets": {
      "ping": "8.8.8.8",
      "dns": "google.com",
      "http": "https://example.com"
    }
  },
  "maintenanceWindow": {
    "enabled": true,
    "startHour": 4, "startMinute": 0,
    "endHour": 4, "endMinute": 10
  },
  "userInfo": {
    "provider": "Magenta",
    "customerId": "08154711",
    "userName": "Max Mustermann"
  },
  "theme": {
    "darkMode": false
  }
```

```
}
```

9.8 Backup und Migration

Fuer ein vollstaendiges Backup genuegt es, zwei Dateien/Verzeichnisse zu sichern:

1. `config.json` – Alle Einstellungen (Ziele, Passwort-Hashes, Theme)
2. `data/` – H2-Datenbank mit allen Messdaten

Zur Migration auf ein anderes System: SignalReport installieren, diese beiden Dateien/Verzeichnisse kopieren und den Dienst starten. Die Datenbank und Konfiguration werden automatisch erkannt.

10 REST-API-Referenz

SignalReport stellt eine REST-API auf Port 4567 bereit, die sowohl vom integrierten Web-Interface als auch von externen Tools genutzt werden kann. Alle Antworten verwenden das JSON-Format.

10.1 Authentifizierung

Wenn die Authentifizierung aktiviert ist, muessen alle API-Aufrufe einen `Authorization`-Header mit HTTP Basic Auth enthalten:

Listing 13: Beispiel: API-Aufruf mit Authentifizierung

```
curl -u admin:meinpasswort http://localhost:4567/api/measurements?
    limit=10
```

Schreibende Endpunkte (POST) erfordern Admin-Rechte.

10.2 Messdaten

Methode	Pfad	Beschreibung
GET	`/api/measurements`	Letzte Messungen abrufen. Parameter: `limit` (Anzahl, Standard: 50)
GET	`/api/statistics`	Statistiken fuer einen Messtyp. Parameter: `type` (PING|DNS|HTTP), `hours` (Zeitraum)
GET	`/api/hourly`	Stuendliche Durchschnittswerte fuer Heatmap. Parameter: `type`, `days`

Tabelle 5: API-Endpunkte: Messdaten

Beispiel-Antwort (`/api/measurements?limit=1`):

```
[{
  "timestamp": "2026-03-15T14:32:05Z",
  "target": "8.8.8.8",
  "latencyMs": 23.45,
  "success": true,
  "type": "PING",
  "localIPv4": "192.168.1.100",
  "externalIPv4": "85.182.xxx.xxx",
  "hostHash": "a3f8b2c1..."
}]
```

10.3 Berichte und Export

Methode	Pfad	Beschreibung
GET	`/api/report`	PDF-Bericht generieren. Parameter: `hours` (24, 168, 8760). Antwort: PDF-Datei (application/pdf)
GET	`/api/csv/export`	CSV-Export aller Messdaten. Antwort: CSV-Datei (text/csv)
GET	`/api/csv/export/filtered`	CSV-Export mit Zeitfilter. Parameter: `hours`

Tabelle 6: API-Endpunkte: Berichte und Export

10.4 DNS-Benchmark

Methode	Pfad	Beschreibung
GET	`/api/dns-benchmark`	DNS-Benchmark starten. Fragt alle konfigurierten DNS-Server parallel ab. 30-Sekunden-Cooldown zwischen Aufrufen.

Tabelle 7: API-Endpunkte: DNS-Benchmark

10.5 IP-Tracking und Host-Info

Methode	Pfad	Beschreibung
GET	`/api/ip-changes`	Letzte IP-Aenderungen. Parameter: `limit`
GET	`/api/ip-change-stats`	Statistik ueber IP-Aenderungen pro Host
GET	`/api/hosts`	Liste aller registrierten Hosts

Tabelle 8: API-Endpunkte: IP-Tracking

Methode	Pfad	Beschreibung
GET	`/api/settings`	Aktuelle Konfiguration abrufen
POST	`/api/settings`	Konfiguration aktualisieren (Admin-Rechte erforderlich)
GET	`/api/theme/settings`	Theme-Einstellungen abrufen
POST	`/api/theme/settings`	Theme aendern. Body: `{``darkMode'': true}`

Tabelle 9: API-Endpunkte: Konfiguration

10.6 Konfiguration

10.7 Authentifizierung und Setup

Methode	Pfad	Beschreibung
POST	`/api/setup/complete`	Ersteinrichtung abschliessen. Setzt Admin-Passwort und optionale Authentifizierung.
POST	`/api/auth/enable`	Authentifizierung aktivieren
POST	`/api/auth/disable`	Authentifizierung deaktivieren
POST	`/api/auth/change-password`	Admin-Passwort aendern

Tabelle 10: API-Endpunkte: Authentifizierung

10.8 Push-Benachrichtigungen

Methode	Pfad	Beschreibung
GET	`/api/push/settings`	Push-Einstellungen abrufen
POST	`/api/push/settings`	Push-Einstellungen aendern (Schwellwert, Anzahl)
POST	`/api/push/subscribe`	Client fuer Push registrieren
GET	`/api/push/poll`	Ausstehende Benachrichtigungen abrufen (Long Polling)

Tabelle 11: API-Endpunkte: Push-Benachrichtigungen

10.9 HTTP-Statuscodes

Code	**Bedeutung**
200	Erfolgreiche Anfrage
302	Weiterleitung (z. B. zum Setup)
401	Authentifizierung erforderlich oder fehlgeschlagen
500	Interner Serverfehler (Details im Response-Body)

Tabelle 12: Verwendete HTTP-Statuscodes